

Evaluation of IPC through Linux Pipes

Patrick DeCabooter and Shaw Liu

1 Introduction

The cost model of a UNIX pipe is usually assumed and not measured. The buffer is said to be sixteen pages, writers block when it fills, and larger writes amortize better. We measured those claims and asked where the time goes. Placement matters more than payload size. Everything ran in a QEMU guest, so the absolute numbers carry hypervisor overhead, and we rely on the comparisons between placements.

2 Apparatus and method

We ran everything on an Ubuntu 24.04 guest with a custom Linux 7.2.4 kernel on arm64, eight vCPUs and 12 GB of RAM, under QEMU with hvf on an Apple M1. The page size is 4 KiB and pipe-max-size is 1 MiB, so the resize sweep runs unprivileged.

We pin the processes with `sched_setaffinity` after the fork, since `taskset` cannot place the two separately. Each configuration runs five times. Noise can only add time, so we report the minimum latency and the maximum throughput, because throughput is a rate.

3 Choosing a clock

Variables. The cause is the clock API. The effects are accuracy and granularity. **Hypothesis.** `clock_gettime` can resolve a single exchange and `gettimeofday` cannot.

We compared `clock_gettime`, `gettimeofday` and `cntvct_el0`, which is this architecture’s version of `rdtsc`. We timed `sleep(5)`, and all three recovered 5.0039 s and agreed within 0.5 ppm, so all three are accurate. The shared 786 ppm excess over five seconds is `sleep` overshooting and not clock error. We then read each clock 10^6 times back to back. Figure 1

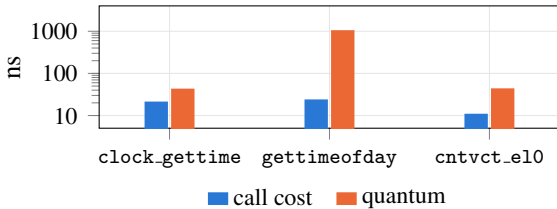


Figure 1: Cost of one call and the smallest nonzero delta, over 10^6 reads.

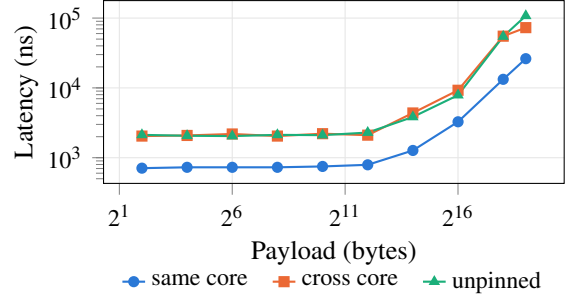


Figure 2: One-way latency against payload size.

shows the result. Call cost does not disqualify any of the three. The quantum disqualifies `gettimeofday`. 97.9 % of its adjacent readings are identical, so it cannot separate two of its own calls or measure a 709 ns exchange. We chose `clock_gettime`, because it shares the quantum of the raw counter and is portable. Neither caveat for cycle counters applies here. The counter is frequency invariant, and it reads within 84 ns across cores.

4 Latency

Variables. The causes are payload size and placement. The effect is one-way latency, taken as half a round trip so that only one clock is involved. **Hypothesis.** Flat while the fixed cost dominates, then linear once the copying dominates.

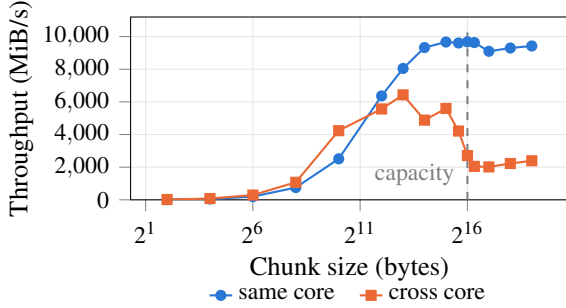
Figure 2 and Table 1 show that shape. The flat region means the cost is per exchange and not per byte. We did not expect the constant offset in that region. The penalty for crossing cores is 1333 ns and it does not depend on the payload. Unpinned runs track the cross-core curve up to 256 KiB, so the scheduler spreads the pair by default. Above that they are worse than either pinning, and we do not know why.

5 Throughput

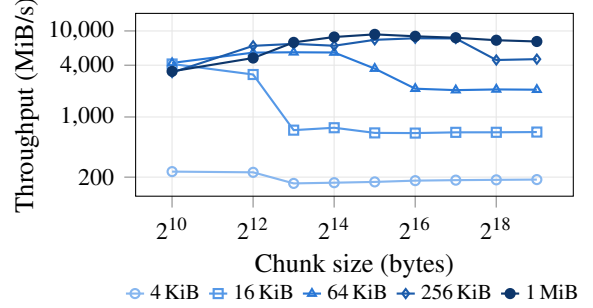
Variables. The causes are chunk size and placement. The effect is sustained one-way bandwidth, acknowledged once for the whole transfer. Each run streams up to 64 MiB, so the acknowledgement is a negligible part of it. **Hypothesis.** Throughput rises as the fixed per-write cost amortizes, then falls once the chunks exceed the capacity.

Size (B)	4	16	64	256	1K	4K	16K	64K	256K	512K
Latency, same core (ns)	709	729	729	729	750	792	1271	3271	13271	26209
Latency, cross core (ns)	2042	2084	2188	2042	2209	2104	4375	9250	55042	72959
Throughput, same core (MiB/s)	13	49	196	752	2511	6366	9329	9678	9301	9425
Throughput, cross core (MiB/s)	19	80	293	1074	4236	5567	4882	2712	2225	2391

Table 1: Measured values at the ten required sizes.



(a) Default capacity. The dashed line marks 64 KiB.



(b) Cross core, capacity set by F_SETPIPE_SZ.

Figure 3: Streaming throughput against chunk size.

Payload	Place	μ s/RT	%usr	%sys	%blk	cs/RT
4 B	same	2.3	0	101	0	1.99
4 B	cross	23.0	3	37	61	2.00
512 KiB	same	58.9	14	88	0	15.99
512 KiB	cross	346.8	0	53	47	15.99

Table 2: Where the wall clock goes, per round trip. User and system sum both processes’ CPU time against one elapsed interval, so they pass 100 % when the two overlap. %blk is the remainder and “cs/RT” is voluntary context switches.

Figure 3a shows the rise, but only the cross-core curve falls. The same-core curve stays flat through chunks eight times the capacity. Blocking cannot be the explanation, since the pipe is the same size in both cases and the same-core writer blocks just as often. The cross-core case loses overlap, because past the capacity the writer stalls mid-chunk.

6 Where the time goes

Variables. The causes are placement and the ratio of payload to capacity. The effects are the user, kernel and blocked split of elapsed time and the context switches per exchange. **Hypothesis.** If the cross-core penalty is a scheduling cost, both placements issue the same calls and the same switches, and the difference appears as time that is neither user nor kernel.

A nonblocking pipe accepts exactly 65,536 bytes before EAGAIN, which matches F_GETPIPE_SZ and the sixteen pages we assumed above. At 4 B both

placements do the same work, and `strace` confirms two `write` and two `read` calls per round trip, but the cross-core pair is ten times slower and most of its wall clock is neither user nor system time. That residue is time spent descheduled while waiting for a wakeup on another core, and it is why the cross-core pair is slower.

The context switch column depends on the capacity. The counts match $2^{\lceil \text{size}/\text{capacity} \rceil}$ against the measured 64 KiB. Figure 3b then makes the capacity the independent variable, and the knee follows the configured size, so we think the capacity is the cause and not just a correlation.

7 Conclusions

Payload size is only one of the two quantities that set pipe cost. The capacity decides how many handoffs a transfer needs. The placement decides what each handoff costs. Spreading a pair across cores never helps latency, and it helps throughput only at small payloads. Everything ran on one machine, so it is hard to tell whether these numbers hold on other hardware. No `perf` build matches this kernel, so Table 2 rests on `getrusage` and `strace`.

Use of AI tools. We used AI assistance to review code, drive the experiment runs, and brainstorm the analysis.